

An Extensive Study on Multi-Priority Algorithm in Test Case Prioritization and Reduction

Longbo, Li

College of Computer and Information Science, Southwest
University, China
lilongbo@email.swu.edu.cn

Yuan, Yuan

College of Computer and Information Science, Southwest
University, China
yy_199801@126.com

Yanhui, Zhou*

College of Computer and Information Science, Southwest
University, China
xiaohui@swu.edu.cn

Shenghua, Wu

College of Computer and Information Science, Southwest
University, China
wush123@email.swu.edu.cn

ABSTRACT

Although test case prioritization and reduction are two different problems in regression testing, they are essentially interrelated. To improve the effectiveness of regression testing, we need to perform fewer test cases, and hope to detect program faults as early as possible. However, most existing techniques haven't proposed a better solution to solve these two issues at the same time. In this paper, we present a multi-priority algorithm combine mutation testing and clustering techniques, and use clustering techniques to put test cases with similar fault-detection ability into a cluster, the multi-priority algorithm selects a high-priority test in each cluster. The results show that the multi-priority algorithm not only reduces a large number of test cases, but also obtains the results of test case prioritization better than greedy algorithm and random reduction method. Especially, the average reduction size of test cases is 40.23% in total 556018 test cases, and removing test cases that trigger real program faults only accounts for 0.7435% of all tests. Our method achieves a greater reduction in the number of test cases at the expense of a mini loss of in fault-detection ability. The effectiveness of test case prioritization is 2.63% higher than other methods. In addition, we find that the different number of clusters affect the effectiveness of test case prioritization and reduction in regression testing.

CCS CONCEPTS

• **Software and its engineering**; • **Software creation and management**; • **Software verification and validation**; • **Software defect analysis**; • **Software testing and debugging**;

KEYWORDS

mutation testing, test case prioritization and reduction, regression testing, clustering

ACM Reference Format:

Longbo, Li, Yanhui, Zhou*, Yuan, Yuan, and Shenghua, Wu. 2021. An Extensive Study on Multi-Priority Algorithm in Test Case Prioritization and Reduction. In *2021 2nd Asia Service Sciences and Software Engineering Conference (ASSE '21)*, February 24–26, 2021, Macau, Macao. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3456126.3456135>

1 INTRODUCTION

Mutation testing is an effective technique for evaluating the quality of test suites in regression testing. It is widely considered to be the most trouble-shooting and rigorous testing standards for other coverage criteria [1]. The mutation testing mainly consists of two phases. In the first phase, according to competent programmer hypothesis [2] and coupling effect [3], using mutation operator generates many mutation programs on Program Under Test (PUT). Every mutation operator makes simple grammatical changes to PUT. Many researchers have proposed various mutation operators [4–6]. In this paper, we only consider the first-order mutation operator, the higher-order mutation operator is to make multiple changes in the PUT. This paper isn't involved the higher-order mutation operator.

In the second phase, test cases perform the mutation testing. If mutant and original program return different results when implementing the same test, then we said the test case *kills* the mutant. To kill mutants, test cases not only need to cover the PUT but also execute the mutated part. In other words, the mutation-based approaches can be constructed at least as strong as coverage-based approaches [7].

The mutation-based test case prioritization and reduction methods have been studied by researchers [8–12]. However, they only considered one aspect: test case prioritization or test case reduction. Although test case prioritization and reduction are two different regression testing issues, they can be related to one another [13]. For example, selecting a set of n test cases from a test suite would be to prioritize the whole set. In other words, we maybe find a optimal sequence in the process of selecting a subset of test suite.

As we know, test cases have the attributes of diversity and similarity. The diversity and similarity of tests have received much attention in test case prioritization and reduction [14–17]. Recently, Donghwan et al. [18] considered the diversity of tests and improved the fault-detection capabilities of test suites in mutation testing. They theoretically and empirically evaluated the advantages of test diversity in improving test suite's fault-detection capability [19].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASSE '21, February 24–26, 2021, Macau, Macao

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8908-2/21/02...\$15.00

<https://doi.org/10.1145/3456126.3456135>

Emilio et al. [16] proposed a similarity-based test suite reduction method that quickly removed numerous test cases.

However, reducing tests based on its similarity hasn't obtain enough notice in mutation testing. We hope to remove numerous test cases while minimizing the loss of fault-detection capabilities of test suites. In the mutation testing, a mutation operator produces many mutants, and these mutation programs have obviously diversity properties. A practical point of view is that certain type of mutants are more important than other mutants [20]. It is reasonable to assume that mutants affect the priority of test cases. Therefore, choosing higher priority test cases is a significant method in test case prioritization. In addition, to reduce vast test cases, clustering technique can cluster test cases with the same fault-detection ability into a cluster. In other words, the fault-detection capabilities of test cases among the cluster are similar. Consequently, we delete abundant test cases in regression testing by picking a high-priority test case in each cluster.

In this paper, we propose a multi-priority algorithm combine mutation testing and Hierarchical Clustering Algorithm(HCA) [21]. The HCA puts together test cases that are similar into a cluster. Test cases are similar if they kill nearly the same mutants. Choosing a high-priority test case among the cluster obtains reduced test suite. Note that clustering test cases needs to perform mutation testing and construct a multi-priority matrix, and detailed method is introduced in Section 3. To select high-priority test cases, we present the concept of mutation program unit priority and change the priority of test cases through high-priority mutation programs. The high-priority mutants and tests are called *edge test* and *edge mutation program*. The *edge test* and *edge mutation program* is different from stubborn mutants. The stubborn mutant can be killed, but none of the existing tests can kill the mutant [22]. In other words, test cases theoretically kill these mutants, but we currently don't find tests that can kill this type of mutants.

To sum up, this paper makes the following contributions:

- The multi-priority algorithm selects high-priority tests, which accomplish test case prioritization and reduction at the same time in regression testing. This paper implements an empirical analysis for test case prioritization and reduction techniques. The multi-priority algorithm can remove 40.23% of all test cases. Especially, in the total of 807 test cases that trigger real program faults, it only deletes 6 test cases, and accounts for 0.7435% of all the test cases that trigger real program faults. Single-objective greedy algorithm and random reduction method delete 33.46% and 36.06% of tests that trigger real program faults, respectively. In addition, the multi-priority algorithm achieves the loss of the test suite's fault-detection capability within 11% when reducing the number of test cases reaches 60%.
- For test case prioritization, this algorithm is 2.63% higher than greedy algorithm in APWFD. The random reduction method performs worse in test case prioritization. Whether test case prioritization or reduction, this algorithm is effective in regression testing.

The paper is structured as follows: In Section 2, we introduce the background. In Section 3, we propose our approach. We describe the empirical evaluation and experimental design in Section 4. In

Section 5, we analysis the experimental results. We conclude our work in Section 6.

2 BACKGROUND

2.1 Test Case Prioritization

The test case prioritization refers to detecting faults as early as possible. The mutation-based test case prioritization involves mutation programs are killed as early as possible. The test case prioritization goal is to find an ideal test case execution order that exposes faults as early as possible.

Many researchers have carried out related research [23–25]. The test case prioritization problem was formally defined by [25].

Definition 1: Test case prioritization problem

Given:

A test suite, TS

The set of permutations of TS , PTS

A function that gives a numerical score for $T' \in PTS$, f

Problem:

Find $T' \in PTS$ such that

$$(\forall T'') (T' \in PTS) (T'' \neq T') [f(T') \geq f(T'')]$$

For Definition 1, PTS represents the all sequences of the given test cases in TS , and f indicates an evaluation function that calculates an award value as an sequence $T' \in PTS$. Of course, we usually use the Average Percentage of Fault Detection (APFD) to assess the results of test case prioritization [26]. Higher APFD means that program faults are detected earlier. It is formalized as follows:

$$APFD = 1 - \frac{TF_1 + \dots + TF_i + \dots + TF_n}{nm} + \frac{1}{2n} \quad (1)$$

The TF_i shows the first test case position in test case sequence among n test cases, which detects the j th fault total m faults. According to mutation program unit priority matrix in Section 3.1, we define the Average Percentage of Weight Fault Detection (APWFD) as follows:

$$APWFD = 1 - \frac{W_1 \times TF_1 + \dots + W_n \times TF_n}{\sum_{i=1}^n W_i \times n} + \frac{1}{2n} \quad (2)$$

For APFD, n and m have the same priority and the values of priority are one. In the APWFD, the weight of mutation program unit m_i is W_i . As for Equation 2, each mutant has different priorities. If the weight of each mutation program is one, then APWFD is converted to APFD. Note that the priority of mutation programs depend on the test suite used. For example, the greedy algorithm selects a test case order $T_2 - T_1 - T_3 - T_0$ in Table 2. The value of APWFD in this sequence is 66.92%. If we consider the edge test and edge mutation program by multi-priority algorithm. This algorithm elects a test case order $T_2 - T_3 - T_1 - T_0$. The value of APWFD is 81.62%. The greedy algorithm is lower 14.7% than the multi-priority algorithm in APWFD.

2.2 Test Case Reduction

The test case reduction doesn't change the mutation score when removing redundant test cases in mutation-based test case reduction. Rothmel et al. [27] formally defined the problem of test case reduction as follows:

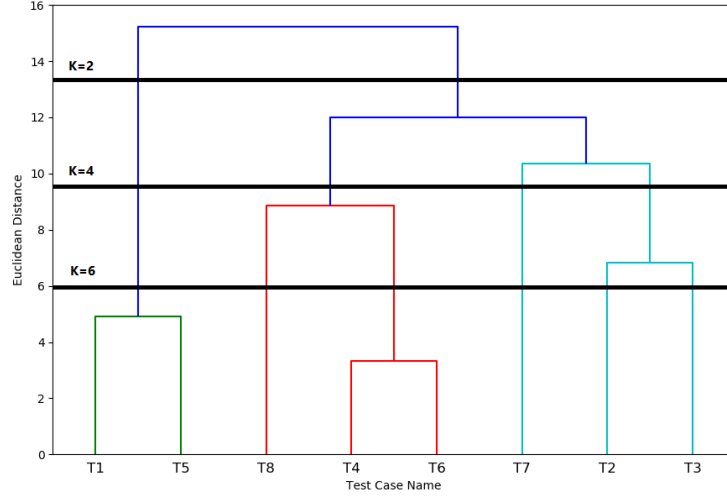


Figure 1: Hierarchical clustering tree

Given: A test suite, T , a set of test requirements $r_1, \dots, r_n$, which must be satisfied to provide the desired adequate testing of the $T, T_1, \dots, T_n$, one associated with each of r_i s such that any one of the test cases t_j belonging to T_i can be used to achieve the requirements r_i .

Problem: Find a representative set, T' , of test case from T that satisfies all r_i s.

A test suite is evaluated by the mutation-adequate, which counts the number of killed mutants. In mutation-based test case reduction, if mutants killed by a single test case are removed, the test suite's mutation score will be fell, and the fault-detection capability loss of test suite has significantly increased. To reduce the loss of test suite's fault-detection capability, we should select test cases that kill a single mutant, which ensure a higher fault-detection ability of reduced test suite. If higher mutation score (this score is only related to the mutation score of the original test suite) is guaranteed, the test suite can find more program faults [28]. Therefore, we think that mutants have diversity attributes and each mutant has different priorities. The mutant killed by a single test case has better higher priority and changes the priority of test cases, we will use this type of mutant to prioritize and reduce test cases in regression testing.

2.3 Cluster Quantity Selection

Tests with the same fault-detection capability are clustered into a cluster by using the HCA. However, if there are a lot of clusters, more tests are selected, then we can't reduce a large number of tests; In contrast, if there are merely a few clusters, few tests are selected and result in high loss of fault-detection capability of the test suite. Consequently, it is necessary to explore the impact of different clusters and distance calculation ways (similarity calculation between clusters) on test case prioritization and reduction.

Figure 1 shows that a hierarchical clustering tree. The HCA does not require a pre-specified number of clusters. However, the number of clusters affect the size of the reduced test cases. In these cases, the HCA needs to be cut at some level. We divide the entire tree,

from top to bottom, according to the distance to divide the cut level. When the distance reaches a threshold, the clustering algorithm is terminated, and we obtain the result of clustering. For the setting of the distance threshold, we are based on the percentage of the maximum distance in the test case similarity matrix. For example, the maximum distance between test cases is 7.5 in Figure 1 (distance calculation way is Euclidean Distance), the cut level-40 represents the termination of clustering when the maximum distance of among clusters reaches 4.5 ($7.5 - (7.5 \times 40\%)$), then the number of clusters is 5 ($k=5$). Assume that there are no redundant test cases in these clusters, our algorithm can pick 5 test cases, the size of reduced test suite is 66.66%. In other words, if the cut level is larger, the threshold will be smaller, the clustering algorithm will generate more clusters, and the number of selected test cases will also increase.

3 PROPOSED APPROACH

This section introduces the proposed method with respect to test case prioritization and reduction in regression testing. First we define the *edge mutation program* and *edge test*. To reduce and prioritize test cases, we construct the mutation program unit priority matrix through the algorithm 2. Clustering test cases by the HCA obtains the results of cluster. We select the high-priority test cases (*edge test*) in each cluster via the algorithm 2.

3.1 Edge Test and Edge Mutation Program

To reduce and prioritize test cases, we need to change the priority of test cases through mutation programs. First we define the *edge mutation program*. Given a test suite TS , and a original program P , After mutation testing, a mutation operator generates a set of mutants mutation program unit.

Definition 2: *edge mutation program*

Given: a mutant $m_i \in MPU$, it is killed by a single test case $t_i \in TS$. We said that m_i is a edge mutation program iff for only $t_i \in TS$, $TS - \{t_i\}$ does not killed the m_i with respect to MPU and TS .

Table 1: Mutation Program Unit Kill Matrix

Test	Mutation Program Unit (MPU)							
	M0	M1	M2	M3	M4	M5	M6	M7
T1	1	0	1	0	0	1	0	0
T2	1	1	1	0	1	0	0	0
T3	0	1	1	1	1	1	0	0
T4	0	0	1	0	0	0	1	1

Table 2: Mutation Program Unit Priority Matrix

Test	Mutation Program Unit (MPU)							
	M0	M1	M2	M3	M4	M5	M6	M7
T1	0.5	0	0	0	0	0.5	0	0
T2	0.5	0.5	0	0	0.5	0	0	0
T3	0	0.5	0	0.75	0.5	0.5	0	0
T4	0	0	0	0	0	0	0.75	0.75

Note that this definition of *edge mutation program* is in the context of a test suite TS and MPU . In addition, we define the *edge test*.

Definition 3: *edge test*

Given: a test case $t_i \in TS$ is *edge test* iff $TS - \{t_i\}$ dose change the original mutation score with respect to TS and MPU . The edge test must be able to kill the *edge mutation program*.

Again, note that this edge test is a subset of the minimal test sets. For the mutation-based minimal test sets, Paul et al. [29] defined the minimal test sets.

Definition 4: minimal test sets

A test set T is minimal iff for any test $t_i \in TS$, $TS - \{t_i\}$ does not maintain the mutation score with respect to M and T .

The M is a set of mutants and the T is a test suite. According to the definition of the MPU , the $MPU \in M$ in this paper.

3.2 Mutation Program Unit

A set of mutants generated by the same mutation operator is called Mutation Program Unit (MPU). Hence, a mutation operator constitutes a mutation program unit, and the MPU contains a lot of mutants. We generate a mutation program unit kill matrix (MPUKM) through the algorithm 2 after mutation testing. Table 1 shows that every m is a mutant in MPUKM.

In Table 1, the 1 represents a test kill a mutant, and 0 means that the test case didn't kill a mutant. For instance, m_2 is killed by all test cases, no matter how we prioritize or reduce test cases, the mutant hasn't an impact on the test suite's mutation score. Obviously, the priority of the mutant is lower than other mutants.

After mutation analysis, we generate the MPUKM in Table 1, and structure the mutation program unit priority matrix (MPUPM) through the algorithm 2. The priority of each mutant is related to a given test suite. For the MPUPM in Table 2, given a PUT, generates m mutants and n test cases, defines a $m \times n$ mutation program unit kill matrix H that represents the results of mutation analysis. H_{ij} indicates whether the test case j kills the mutant i . We can formalize

this mutation program unit priority weight as W_i :

$$W_i = -\frac{\sum_{j=1}^n H_{ij}}{n} + 1 \quad (3)$$

Based on Table 1, we use equation 3 to calculate the weight of each mutant. For example, there are four tests and m_0 is killed by two tests. Thus, the weight of m_0 is 0.5. We update all the weight of m and get Table 2. The m_2 is a redundant mutant by optimizing the weight calculation. Because of m_2 will must be killed no matter how we prioritize test cases. The *edge mutation program* is m_2 , m_6 and m_7 . The *edge test* includes T_2 and T_3 in Table 2

3.3 The Multi-Priority Algorithm

Based on the analysis of Section 3.1 and 3.2, we introduce the multi-priority algorithm. This algorithm combines mutation testing and hierarchical clustering techniques.

First of all, we use major [30] to perform mutation analysis on the PUT and generate the MPUPM from the results of mutation analysis. The algorithm 2 shows how to generate the MPUPM. The algorithm takes the T is a test suite, a set of mutant logs, a *testMap* and a *killMap* are obtained through mutation analysis tool Major.

In algorithm 2, M_i represents a set of mutants. K represents test case kill information. For instance, Lines 9-19 traverse each mutant and obtain which tests kill it. Update each mutant's weight via the Equation 3 in lines 20-24. To calculate the similarity of test cases, we convert the dataframe into a matrix.

The algorithm 2 shows how to generate a prioritized and reduced test case sequence. The algorithm takes the MPUPM as inputs, and return a prioritized and reduced test case sequence. In algorithm 2, to calculate the percentage of test cases that trigger real program faults, line 1-2 obtain test case number in original test suite. Line 3 shows that distance calculation function `max_dist()`. Note that we use six distance ways (Canberra Distance, Chebyshev Distance, Euclidean Distance, Hamming Distance, Manhattan Distance and Squeclidean Distance) to calculate the `max_dist()`. Due to we select

the high-priority test cases in each cluster, different distance calculation ways mainly affect the results of clustering, thus we sort the test cases by the value of weight in each cluster. The lines 4-8 show that how to generate clusters based on the setting threshold. We select minimal distance paired test cases, put the closest test cases into the same clustering one by one and constitute a new clustering. When the distance of between each cluster reaches the threshold, and program returns clustering results. The lines 9-13 show that pick on *edge test* in all clusters. Every test has a priority weight, we choose the *edge test* and receive prioritized and reduced test case orders. In particular, if a test case number isn't in the original test suite means that our algorithm removes test cases that trigger real program faults in lines 14-16. In addition, lines 14-16 can calculate the position of the test cases that trigger real program faults in a sequence of test cases. For *Position Effectiveness*, the evaluation metric is explained in Section 4.1.

Algorithm 1 A mutation program priority matrix

Input: Mutant log, testMap, killMap, T

Output: A mutation program priority matrix

```

1: Reading a mutant log  $M_i$ ;
2:  $M[] \leftarrow M_i$ ;
3: Reading a testMap and a killMap file;
4:  $T\{name, Tid\} \leftarrow testMap$ ,  $TS[name] \leftarrow testMap$ ;
5: Reading a killMap file;
6:  $K\{name, Mid\} \leftarrow killMap$ ;
7: while ( $TS[]$  not null) do
8:   If  $T[]$  in  $K$  then
9:     for  $M[]$  not null do
10:      if  $K$  contains  $Mid$  then
11:        test case kills a mutant;
12:        label(1);
12:      test case doesn't kill a mutant;
13:      label(0);
14:    end
15:  for  $M[]$  not null do
16:    the test case does not kill any mutants;
17:    label(0);
18:  end
19: end
20: for dataframe not null do
21:  calculate each column weight;
22:  if  $W_i$  not 0 then
23:    update each column data;
24: end
25: return A mutation program priority matrix;
```

4 EMPIRICAL EVALUATION AND EXPERIMENTAL DESIGN

4.1 Empirical Evaluation

For empirical evaluation, we investigate the following two research questions:

1. **RQ1:** What are the advantages of multi-priority algorithm compared with other methods in test case prioritization and reduction?

2. **RQ2:** How does the number of different clusters effect the fault detection capabilities of test suites?

For **RQ1**, we mainly study the advantages of the multi-priority algorithm in test case prioritization and reduction. We will evaluate the *Test Effectiveness*, *Position Effectiveness*, *APWFD* and the reduction rate of test cases that trigger real program faults (these faults are different from mutants, they are real faults discovered by program developers). In addition, many test case reduction methods are evaluated by manual injection faults or mutants. It makes sense to investigate the reduction of test cases that trigger real program faults in regression testing. However, in actual test activities, we don't know which test cases can detect real bugs in advance. Consequently, we use a open source benchmark data set *Defects4J* [31]. It contains the bug program version and the fixed program version for every PUT. Making use of test cases that trigger real program faults can better evaluate our proposed method, which shows the effectiveness of multi-priority algorithm in regression testing.

RQ2 mainly explores the impact of different clusters on test case prioritization and reduction. If a cluster is too large, it maybe contains many redundant test cases in the cluster. If it is small, tests with the same fault-detection ability aren't added to a cluster, which affects the results of test case prioritization and reduction. In this research, we evaluated 354 test suites range from 5 to 8048 test cases. It is worth to consider to adapt a new cluster size not only in test case prioritization but also in test case reduction. To obtain better test case prioritization and reduction results, it is necessary to perform an empirical analysis in regression testing.

To answer the above questions, we measure the following evaluation metrics. We usually use the mutation score to estimate the fault-detection capabilities of test suites in the mutation-based test case reduction. According to [28], the formalization of the mutation score is described as follows:

$$Mutation\ Score = \frac{Kill_m}{All_m - R_m} \quad (4)$$

The All_m is total mutants after the mutation analysis, R_m represents redundant mutants, and $Kill_m$ indicates mutants are killed by test cases. As we know, it is obviously not comprehensive enough to evaluate the fault-detection capabilities of test suites if only consider the mutation score. The test cases that trigger real program faults are an important evaluation metric. It can display the real fault-detection capabilities of test suites. In addition, to measure the effectiveness of test suites, we describe the *Test Effectiveness* (TE) according to [32]:

$$Test\ Effectiveness = \frac{No.of\ Test\ Case}{No.of\ Exposed\ Faults} \quad (5)$$

For Equation 5, we use mutants to represent the exposed faults in test case reduction. For instance, fewer tests expose more faults means better the effectiveness of the tests. The *Test Effectiveness* has been widely used in the literature [1, 33, 34]. We also calculate the position of test cases that trigger real program faults in the entire test case sequences. The use of test cases that trigger real program faults in regression testing is of great significance in showing the fault-detection capabilities of test suites. The formalization of the

Position Effectiveness is described as follows:

$$\text{Position Effectiveness} = \frac{\text{The Position of Test case}}{\text{The Number of Test case}} \quad (6)$$

The *Position Effectiveness* is the evaluation metric. If test cases that trigger real program faults discover faults earlier, it is worth to consider to adapt the new evaluation metric not only in test case prioritization but also test case reduction. For example, the value of *Position Effectiveness* represents detection faults earlier in test case orders. In addition, the *Position Effectiveness* is different from the *APWFD*. It is only affected by different test sequences.

Algorithm 2 A prioritized and reduced test case sequence

Input: Mutation program unit priority matrix

Output: A prioritized and reduced test case sequence

```

1: Reading original test suite;
2:  $\text{bug\_dict}\{\} \leftarrow T$ ;
3:  $\text{max\_dist}()$ ;
4: while true do
5:   calculate the maximum distance;
6:    $\text{threshold} \leftarrow \text{max\_dist}()$ ;
7:   HCA // setting threshold;
8:    $\text{new\_cluster} \leftarrow \text{HCA}$ ;
9: end
10: while true do
11:   // sort intra-cluster, prevent redundant tests;
12:   for ( $\text{new\_cluster not null}$ ) do
13:     // select high-priority tests;
14:      $\text{ADD}(\text{LIST}[i])$ ;
15:   end
16: end
17: while ( $\text{list not null}$ ) do
18:   if  $\text{list}_i$  in  $\text{bug\_dict}$  then
19:      $\text{bug\_list}(i) \leftarrow \text{list}$ ;
20: end
21: return a prioritized and reduced test case sequence
```

The test suite reduction rate (RS) and fault-detection capability (RF) are commonly used in these empirical evaluations [13, 16]. T and T_r are the size of the original and reduced test suites. Consistently, F is the number of faults detected by the original test suite, while the F_r is the number of faults detected by the reduced test suite. RS is defined as follows:

$$\text{RS} = \frac{\text{No.of } T - \text{No.of } T_r}{\text{No.of } T} \quad (7)$$

The value of RS ranges from 0 to 1. RS represents the number of test cases within the original test suite that are not present in the reduced one. If RS is 1 means that the size of the original test suite already has been reduced. Thus, a high value for RS is hopeful. However, note that the higher RS does not the better, we also need to consider the fault-detection capability of the test suite. RF is defined as follows:

$$\text{RF} = \frac{\text{No.of } F - \text{No.of } F_r}{\text{No.of } F} \quad (8)$$

Similarly, the value of RF ranges from 0 to 1. For RF, it is the percentage of faults that the reduced test suite does not detect with

respect to the faults the original test suite detects. We hope to get lower the RF. Generally speaking, the RS represents the reduction degree of test suite, and RF shows the fault-detection capabilities of reduced test suites. In addition, the APWFD and *Position Effectiveness* indicate the effectiveness of test case prioritization.

4.2 Experiment Design

We consider the open project Defects4J [31] data set, which contains a large number of developer-written test cases, as well as test cases that trigger real program faults. Specially, it includes both faulty and fixed versions for each program. Furthermore, we use the Major [30] to perform mutation analysis. To cluster the test case, we perform the HAC. It's convenient to select for cluster size in test suite reduction.

We select 354 fixed programs and perform mutation analysis via the Major. Moreover, Major contains many mutation operator and utilizes total mutation operators that include binary operator replacement (BOR), unary operator replacement (UOR), constant value replacement (CVR), branch condition manipulation (BCM), and statement deletion (CD). The Figure 2 shows that total experimental setup. We configure Defects4J in Ubuntu18.04 LTS (intel i7 cpu, RAM 16G). Hence, we perform mutation analysis on the fixed version to gain mutation program kill information, and utilize the bug version to measure the effectiveness of our methods. For the evaluation result of test case prioritization, three methods (multi-priority algorithm, greedy algorithm and random reduction method) select the same number of test cases. Using greedy algorithm and random reduction method selects a contrast test suite. And finally, we evaluate the results of test case prioritization and reduction.

5 EXPERIMENT RESULTS AND ANALYSIS

5.1 RQ1: What are the advantages of multi-priority algorithm compared with other methods in test case prioritization and reduction?

For RQ1, we aim to investigate the advantages of the multi-priority algorithm in test case prioritization and reduction. We calculate the *Test Effectiveness*, *Position Effectiveness*, APWFD, RS and the reduction rate of test cases that trigger real program faults.

We computed descriptive statistics and used graphical representations to summarize data (i.e., boxplots). In Figure 3 (a), the multi-priority algorithm is higher 19.34% than random reduction method in average percent of APWFD. Consistently, the multi-priority algorithm is higher 2.63% than greedy algorithm in average percent of APWFD.

We usually evaluate the results of test case prioritization via the calculation of APWFD. However, in order to better highlights the advantages of the multi-priority algorithm in test case prioritization, we calculate the proportion of test case locations in total test case sequences. Each test suite contains at least a test that triggers real program faults. If a program contains numerous test cases that trigger real program faults, we measure the average position of test cases.

In the Figure 3 (b), our algorithm retains more test cases that trigger real program faults. The greedy algorithm and random reduction method remove a large number of test cases that trigger

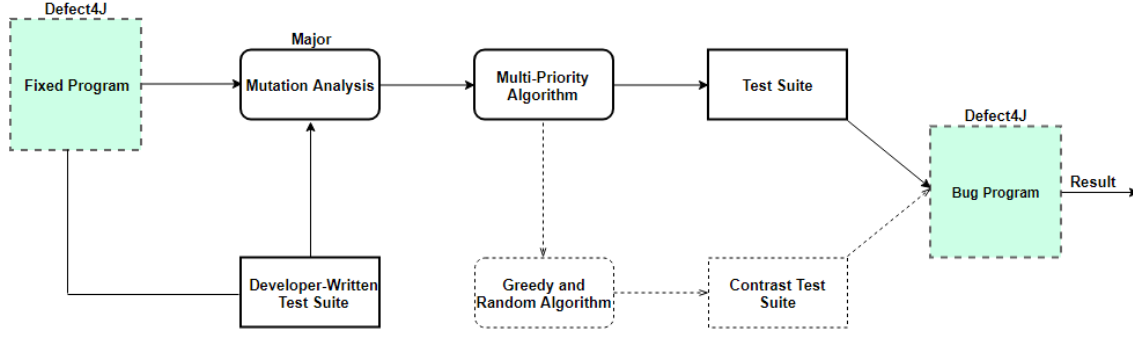


Figure 2: Experiment setup

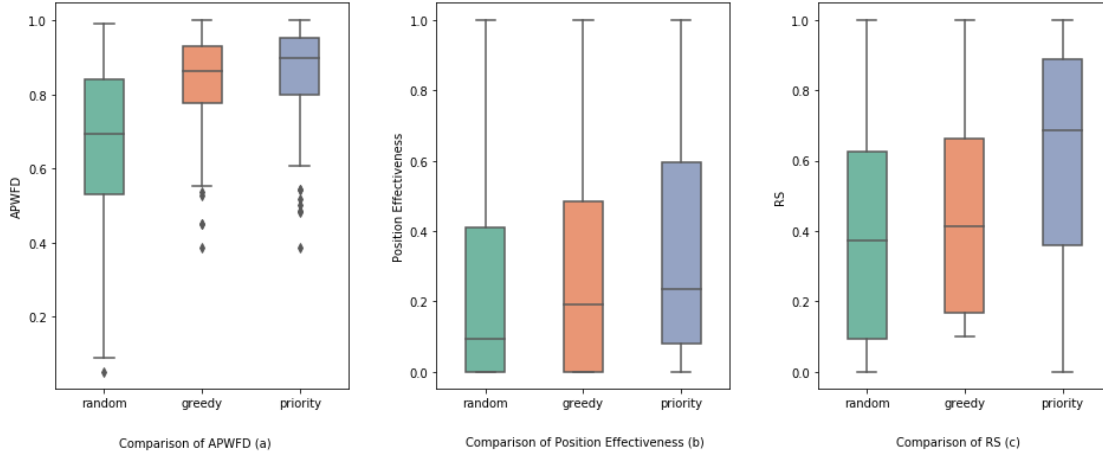


Figure 3: Boxplots of three algorithms for APWFD, Position Effectiveness and RS

real program faults. The Figure 3 (c) shows that the x coordinate presents the three methods we used, the y represents the number of test cases within the original test suite that are not present in the reduced one. A high value for RS is hopeful. The multi-priority algorithm is higher than greedy algorithm and random reduction method in RS. This algorithm can ensure that the fault-detection capability loss of the test suite is within 11% when the average reduction number of test cases reach 60%.

To compare the advantages of the multi-priority algorithm in tests that trigger real program faults, we also statistic the results of removing tests from all test suites. The Table 3 shows that the multi-priority only removes six test cases and the reduction rate is 0.7435%. For the greedy algorithm, it is obviously unable to ensure the lower fault-detection capability loss of test suite. Because of it removes 33.46% test case that trigger real program faults and doesn't perform well in test case reduction. In addition, the random reduction method isn't a good choice on test case reduction. The 291 test cases that trigger real program faults are deleted, which account for 36.06% of all tests. In Table 3, the original *Test Effectiveness* is 7.48, the multi-priority algorithm ensures higher fault-detection

Table 3: The Reduced Results of Test Cases that Trigger Real Program Faults

Methods	Statistical Indicators			
	tests	removed	rate	Test Effectiveness
Random	807	291	36.06%	7.48/5.96
Greedy	807	270	33.46%	7.48/3.38
Priority	807	6	0.7435%	7.48/2.72

capabilities when reducing a large number of test cases. This algorithm is higher 8.82% than greedy algorithm in *Test Effectiveness*.

5.2 RQ2: How does the number of different clusters effect the results of test case prioritization and reduction?

Different distance calculation methods can affect the results of test reduction [15], hence, for RQ2, to study the impact of different distance calculation methods on test case prioritization and reduction,

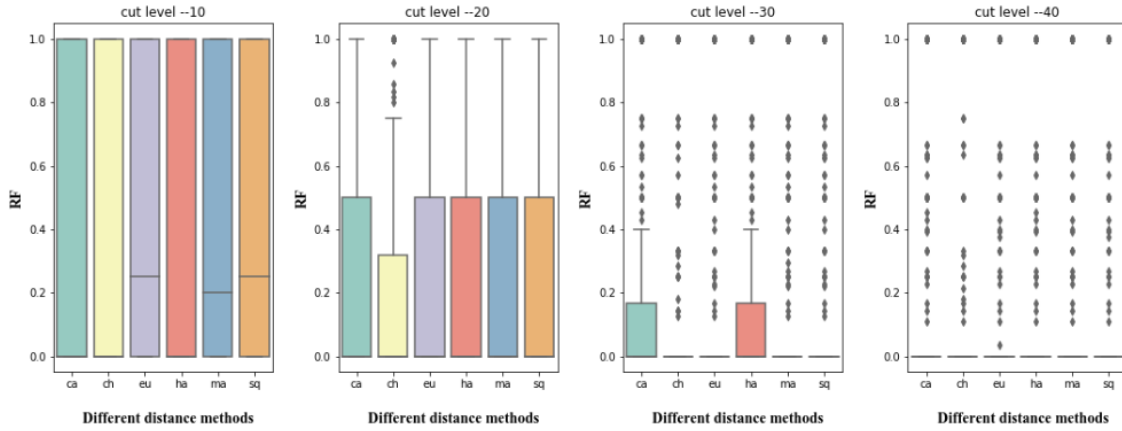


Figure 4: The number of different clusters effect the fault-detection capabilities of test suites (RF).

we use six distance calculation methods to cluster test cases. Note that the six different distance calculation methods are Canberra Distance (ca), Chebyshev Distance (ch), Euclidean Distance (eu), Hamming Distance (ha), Manhattan Distance (ma) and Squeclidean Distance (sq). The loss of fault-detection capabilities of test suites is 10.91% when selects the cut level 40, and the reduction rate of test cases can reach 61.58%. In this paper, we only statistic the cut level 10 to cut level 40. If we continue to increase the cut level, and the number of generated clusters is increasing, and the selected test cases are also increasing. The optimal result of the multi-priority algorithm is to reduce 40.23% of the test cases, the reduction rate of test cases that trigger real program faults is only 0.7435%. In future research, we will optimize this algorithm so that the reduction rate of test cases that trigger real program faults at cut level 40 is within 1%.

To evaluate RQ2, we not only use boxplots to summarize data but also perform a no-parametric statistical hypothesis test. We execute the following four null hypotheses:

- (1) **RF_nh1:** There is no statistically significant difference in the RF values computed by applying Cut-level_ca, Cut-level_ch, Cut-level_eu, Cut-level_ha, Cut-level_ma and Cut-level_sq.
- (2) **RF_nh2:** There is no statistically significant difference in the RF values computed by applying Cut level-10, Cut level-20, Cut level-30 and Cut level-40.
- (3) **APWFD_nh1:** There is no statistically significant difference in the APWFD values computed by applying Cut-level_ca, Cut-level_ch, Cut-level_eu, Cut-level_ha, Cut-level_ma and Cut-level_sq.
- (4) **APWFD_nh2:** There is statistically significant difference in the APWFD values computed by applying Cut level-10, Cut level-20, Cut level-30, Cut level-40.

For RQ2, we use a Kruskal-Wallis rank sum test, and if a null hypothesis is accepted, then we can assume that there is not a statistically significant difference in RF values computed by the six distance methods in test case prioritization and reduction. We used a significance level of 5% the null hypothesis to decide the differences

in the RF values are not statistically significant. In addition, for the APWFD, we used the same statistical method.

In Figure 4, we show the boxplots of the RF values of the four cut levels obtained by using six different distance calculation methods. The boxplots obtained by applying different cut levels suggest that there is a large difference among the RF values. The Chebyshev Distance is a small better than other calculation methods. However, this does not mean that the different distance calculation methods have an impact on the test case prioritization and reduction.

In Table 6, we execute the descriptive statistics (i.e., minimum, maximum, median, mean and standard deviation) among RF and APWFD on each cut level. We know that the data changes within each cut level are not obvious. However, the gap among different cut levels is extremely obvious, the average of RF dropped from 40.67% to 13.26%. Note that some of the programs in Defects4J merely include a test case that triggers real program faults, once this test case is removed, the loss of the fault-detection capabilities of test suites is pretty high. In regard to the APWFD, the different number of clusters can affect the results of test case prioritization. For each cut level, the results of APWFD obtained by using Chebyshev Distance are slightly better than other distance calculation ways, but this advantage doesn't exceed 5%. Consequently, we conclude that there is no a single distance calculation method better than others on APWFD.

For Table5, we report the p-values the Kruskal-Wallis test among each null hypothesis. We calculated the p-values of RF_nh1 and APWFD_nh1. All of p-values are higher than $\alpha(0.05)$. The showed p-values suggests that we can accept RF_nh1 and APWFD_nh1 when using the six different distance calculation methods. In this experimental, there is no statically differences with respect to using the six different distance calculation methods in test case prioritization and reduction. In addition, Smaller standard deviation means that the data is more stable and closer to the average of results in Table 6.

For RF_nh2 and APWFD_nh2, we calculated all cut levels for RF and APWFD, and all cut levels of each distance calculation methods are taken as a p-value. In Table 5, the results of the p-values the

Table 4: The Results of Kruskal-Wallis Test for Different Distance Calculation Methods for p-value

Hypothesis	Statistical cut levels			
	Cut level-10	Cut level-20	Cut level-30	Cut level-40
RF_nh1	0.0674	0.1996	0.6467	0.5093
APWFD_nh1	0.0714	0.2427	0.8783	0.9784

Table 5: The Results of Kruskal-Wallis Test for Different Clusters for p-value

Hypothesis	Ca	ch	eu	Ha	ma	sq
RF_nh2	1.62E-19	1.37E-13	3.33E-23	1.67E-19	4.03E-27	5.88E-24
APWFD_nh2	4.62E-06	0.0143	4.30E-04	0.0130	0.0030	2.14E-05

Table 6: Descriptive Statistics (i.e., minimum, maximum, median, mean and standard deviation) for RF and APWFD in each cut level

		Cut level-10						Cut level-20					
		ca	Ch	eu	ha	ma	sq	Ca	ch	eu	ha	ma	Sq
		Min	0	0	0	0	0	0	0	0	0	0	0
RF	Max	1	1	1	1	1	1	1	1	1	1	1	1
	Median	0	0	0.25	0	0.2	0.25	0	0	0	0	0	0
	Mean	0.413	0.321	0.430	0.416	0.425	0.430	0.267	0.209	0.261	0.267	0.269	0.261
	SD	0.452	0.435	0.458	0.451	0.456	0.458	0.401	0.377	0.400	0.401	0.407	0.400
APWFD	Min	0.372	0.378	0.371	0.371	0.371	0.371	0.455	0.455	0.455	0.455	0.455	0.455
	Max	0.995	0.995	0.995	0.995	0.995	0.995	0.999	0.999	0.999	0.999	0.999	0.999
	Median	0.645	0.687	0.638	0.644	0.641	0.638	0.723	0.735	0.726	0.723	0.720	0.723
	Mean	0.643	0.687	0.644	0.643	0.641	0.645	0.698	0.708	0.700	0.698	0.698	0.700
	SD	0.156	0.141	0.152	0.156	0.155	0.152	0.190	0.188	0.191	0.190	0.191	0.191
		Cut level-30						Cut level-40					
		ca	Ch	eu	ha	ma	sq	Ca	ch	eu	ha	ma	Sq
	Min	0	0	0	0	0	0	0	0	0	0	0	0
RF	Max	1	1	1	1	1	1	1	1	1	1	1	1
	Median	0	0	0	0	0	0	0	0	0	0	0	0
	Mean	0.196	0.157	0.182	0.196	0.177	0.178	0.143	0.109	0.134	0.143	0.133	0.131
	SD	0.365	0.338	0.354	0.365	0.351	0.351	0.318	0.291	0.310	0.318	0.312	0.307
APWFD	Min	0.565	0.565	0.565	0.565	0.565	0.565	0.636	0.629	0.636	0.636	0.636	0.636
	Max	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999
	Median	0.858	0.872	0.874	0.879	0.875	0.865	0.904	0.931	0.923	0.920	0.914	0.910
	Mean	0.826	0.842	0.844	0.845	0.846	0.846	0.895	0.916	0.912	0.909	0.905	0.903
	SD	0.113	0.110	0.111	0.111	0.112	0.112	0.070	0.066	0.065	0.065	0.066	0.066

Kruskal-Wallis test are below $\alpha(0.05)$. On the basis of the observed results, we know that the RF and APWFD are different in cut level by using descriptive statistics and the boxplots. The reported p-values suggests that we can reject RF_nh2 and APWFD_nh2 when considering the different number of clusters in test case prioritization and reduction. There is statistically significant difference in the RF and APWFD values computed by applying the different number of clusters. In other words, the number of different clusters effect the results of test case prioritization and reduction.

6 CONCLUSION

This paper studies the mutation-based test case prioritization and reduction techniques. In previous research, many researchers believed that mutation programs had the same priority in mutation testing. We present the concept of mutation program unit priority. The high-priority mutation program not only modifies test priorities, but selects more mutation-adequate test suites. They have better results compare greedy algorithm and random reduction method in terms of reducing and prioritizing test cases. In particular, we define the edge test and edge mutation program. It

can be well applied to prioritization and reduction of test cases in regression testing.

In this paper, we provide an empirical evaluation of the test case prioritization and reduction techniques. The results show that the reduction rate of test cases is 40.23%. When we use greedy algorithm and random reduction method to reduce test cases, the loss of the fault detection abilities of test suites that trigger real program faults is as high as 33.46% and 36.06%, respectively. While the loss of the fault detection capabilities of test suites is only 0.7435% by the multi-priority algorithm. In addition, we investigate the different number of clusters effect the results of test case prioritization and reduction. we find that the different distance calculation methods have no impact on the results of test case prioritization and reduction.

In the future work, the multi-priority algorithm will be used to reduce mutants and test cases. Hence, we mainly do two aspects of work. On the one hand, we will provide an empirical evaluation of the multi-priority mutation adequacy criterion in terms of improving fault detection capability of test suites. On the other hand, we will take use of the mutant program unit priority to reduce the large number of mutants and test cases. Our goal is to better promote the application of mutation testing in regression testing.

REFERENCES

- [1] J. H. Andrews, L. C. Briand, Y. Labiche, and A. S. Namin. 2006. Using Mutation Analysis for Assessing and Comparing Testing Coverage Criteria. *IEEE Transactions on Software Engineering* 32, 8 (08 2006), 608–624
- [2] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer* 11, 4 (April 1978), 34–41
- [3] A. Jefferson Offutt. 1992. Investigations of the Software Testing Coupling Effect. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (Jan. 1992), 5–20
- [4] P. Anbalagan and T. Xie. 2008. Automated Generation of Point cut Mutants for Testing Point cuts in AspectJ Programs. In 2008 19th International Symposium on Software Reliability Engineering (ISSRE). 239–248
- [5] D. B. Brown, M. Vaughn, B. Liblit, and T. Reps. 2017. The Care and Feeding of Wild-caught Mutants. In Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017). ACM, New York, NY, USA, 511–522
- [6] A. Estero-Botaro, F. Palomo-Lozano, and I. Medina-Bulo. 2010. Quantitative Evaluation of Mutation Operators for WS-BPEL Compositions. In 2010 Third International Conference on Software Testing, Verification, and Validation Workshops. 142–150
- [7] D. Shin, S. Yoo, M. Papadakis, and D. H. Bae. 2019. Empirical evaluation of mutation-based test case prioritization techniques. *Software Testing, Verification and Reliability* 29, 1-2 (2019), e1695
- [8] M. A. Alipour, A. Shi, R. Gopinath, D. Marinov and A. Groce. 2016. Evaluating Non-adequate Test-case Reduction. In Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE 2016). ACM, New York, NY, USA, 16–26
- [9] D. Hao, L. Zhang, X. Wu, H. Mei, and G. Rothermel. 2012. On-demand test suite reduction. In 2012 34th International Conference on Software Engineering (ICSE). 738–748
- [10] L. Li, Y. Zhou, Y. Yu, F. Zhao, S. Wu and Z. Yang. 2019. An Empirical Study of Mutation-Based Test Case Clustering Prioritization and Reduction Technique. *ICSEA 2019* (2019), 13
- [11] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. L. Traon and M. Harman. 2019. Chapter Six-Mutation Testing Advances: An Analysis and Survey. *Advances in Computers*, Vol. 112. Elsevier, 275 – 378
- [12] M. P. Usaola, P. R. Mateo, and B. P. Lamancha. 2012. Reduction of Test Suites Using Mutation. In *Fundamental Approaches to Software Engineering*, Juan de Lara and Andrea Zisman (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 425–438
- [13] S. Yoo and M. Harman. 2012. Regression Testing Minimization, Selection and Prioritization: A Survey. *Softw. Test. Verif. Reliab.* 22, 2 (March 2012), 67–120
- [14] T. Y. Chen, F. Kuo, R. G. Merkel, and T. H. Tse. 2010. Adaptive Random Testing: The ART of Test Case Diversity. *J. Syst. Softw.* 83, 1 (Jan. 2010), 60–66
- [15] C. Coviello, S. Romano, G. Scanniello, A. Marchetto, and A. Corazza. 2018. Clustering support for inadequate test suite reduction. In 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)
- [16] E. Cruciani, B. Miranda, R. Verdecchia, and A. Bertolino. 2019. Scalable Approaches for Test Suite Reduction. In 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). 419–429
- [17] R. Feldt, S. Poulding, D. Clark, and S. Yoo. 2016. Test Set Diameter: Quantifying the Diversity of Sets of Test Cases. In 2016 IEEE International Conference on Software Testing, Verification and Validation (ICST). IEEE Computer Society, Los Alamitos, CA, USA, 223–233
- [18] D. Shin, S. Yoo, and D. Bae. 2016. Diversity-Aware Mutation Adequacy Criterion for Improving Fault Detection Capability. In 2016 IEEE Ninth International Conference on Software Testing, Verification and Validation Workshops (ICSTW). 122–131
- [19] D. Shin, S. Yoo, and D. Bae. 2018. A Theoretical and Empirical Study of Diversity-Aware Mutation Adequacy Criterion. *IEEE Transactions on Software Engineering* 44, 10 (Oct 2018), 914–931
- [20] Y. Jia and M. Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5 (Sep 2011), 649–678
- [21] C. Manning, P. Raghavan, and H. Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press
- [22] X. Yao, M. Harman, and Y. Jia. 2014. A Study of Equivalent and Stubborn Mutation Operators Using Human Analysis of Equivalence. In Proceedings of the 36th International Conference on Software Engineering (ICSE 2014). Association for Computing Machinery, New York, NY, USA, 919–930
- [23] H. Do, G. Rothermel, and A. Kinneer. 2006. Prioritizing JUnit Test Cases: An Empirical Assessment and Cost-Benefits Analysis. *Empirical Software Engineering* 11, 1 (01 Mar 2006), 33–70
- [24] J. A. Jones and M. J. Harrold. 2003. Test-suite reduction and prioritization for modified condition/decision coverage. *IEEE Transactions on Software Engineering* 29, 3 (03 2003), 195–209
- [25] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold. 2001. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering* 27, 10 (10 2001), 929–948
- [26] S. Elbaum, A. G. Malishevsky, and G. Rothermel. 2002. Test case prioritization: A family of empirical studies. *IEEE Transactions on Software Engineering* 28, 2 (02 2002), 159–182
- [27] G. Rothermel, M. J. Harrold, J. Ronne, and C. Hong. 2002. Empirical studies of test-suite reduction. *Software Testing, Verification and Reliability* 12, 4 (2002), 219–249
- [28] M. Papadakis, D. Shin, S. Yoo, and D. Bae. 2018. Are Mutation Scores Correlated with Real Fault Detection? A Large Scale Empirical Study on the Relationship Between Mutants and Real Faults. In 2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE). 537–548
- [29] P. Ammann, M. E. Delamaro, and J. Offutt. 2014. Establishing Theoretical Minimal Sets of Mutants. In 2014 IEEE Seventh International Conference on Software Testing Verification and Validation. 21–30
- [30] R. Just. 2014. The Major Mutation Framework: Efficient and Scalable Mutation Analysis for Java. In Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA 2014). ACM, New York, NY, USA, 433–436
- [31] R. Just, D. Jalali, and M. D. Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA 2014). ACM, New York, NY, USA, 437–440
- [32] M. Papadakis and N. Malevris. 2010. An Empirical Evaluation of the First and Second Order Mutation Testing Strategies. In 2010 Third International Conference on Software Testing, Verification, and Validation Workshops. 90–99
- [33] N. Li, U. Praphamontripong, and J. Offutt. 2009. An Experimental Comparison of Four Unit Test Criteria: Mutation, Edge-Pair, All-Uses and Prime Path Coverage. In Proceedings of the IEEE International Conference on Software Testing, Verification, and Validation Workshops (ICSTW '09). IEEE Computer Society, Washington, DC, USA, 220–229
- [34] E. J. Weyuker. 1993. More experience with data flow testing. *IEEE Transactions on Software Engineering* 19, 9 (09 1993), 912–919.